



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3133 HIGH PERFORMANCE DATA PROCESSING

SECTION 01

PROJECT 2

LECTURER: MOHD SHAHIZAN BIN OTHMAN

SHOPPING

Student Name	Matric No.
TAN YI YA	A23CS0187
NURUL ADRIANA BINTI KAMAL JEFRI	A23CS0258
TEH RU QIAN	A23CS0191

Table of Content

1.0 Introduction	3
2.0 Background	3
2.1 Objectives	3
2.2 Project scope	3
3.0 Data Acquisition and Preprocessing	4
3.1 Data Sources and Collection	4
3.2 Data Preprocessing	4
4.0 Sentiment Model Development	5
4.1 BERT Model	5
4.1.1 Model Choice	5
4.1.2 Training Process	5
4.1.3 Evaluation	6
4.2 CNN Model	8
4.2.1 Model Choice	8
4.2.2 Training Process	9
4.2.3 Evaluation	10
4.3 Model Comparison	11
5.0 Apache System Architecture	12
6.0 Analysis and Results	14
6.1 Key Findings	14
6.2 Visualization	15
7.0 Optimization and Comparison	21
8.0 Conclusion and Future Work	22

1.0 Introduction

The rapid development of information technology has significantly increased the amount of user-generated data content online, such as reviews, comments and social media posts. These data provides valuable information about the public opinion, experience and preferences of a certain product, service or destination. In the tourism industry, online reviews play a crucial role in influencing travellers' decisions and shaping the reputation of tourist attractions. Malaysia, being one of the major tourism destinations in Southeast Asia, receives thousands of reviews from both local and international visitors through platforms such as Google Maps. These reviews contain useful information regarding visitor satisfaction, service quality and overall experiences. However, manually analysing a large number of reviews is time consuming. Hence, sentiment analysis is required to extract opinion and emotions from the enormous amount of textual data.

This project aims to develop a real-time sentiment analysis system for Google Maps reviews of popular tourist attractions in Malaysia. The system collects reviews through web scraping, preprocesses the textual data using natural language processing (NLP) techniques and classifies the reviews into positive, neutral and negative sentiments using machine learning models. To support real-time processing, Apache Kafka is employed as the streaming platform, while a Convolutional Neural Network (CNN) model performs sentiment classification. The analysis results are stored in Elasticsearch and visualised through Kibana dashboards to provide meaningful insights into visitor sentiments and system performance.

2.0 Background

2.1 Objectives

The objectives of this project are:

1. To collect English-language Google Maps reviews from popular tourist attractions in Malaysia using automated web scraping.
2. To preprocess and label the collected review data using a pre-trained transformer-based sentiment analysis model.
3. To train and evaluate both deep learning and traditional machine learning models for three-class sentiment classification (positive, negative and neutral)
4. To design and implement a real-time sentiment analysis pipeline capable of automatically ingesting, classifying and storing Google Maps reviews.
5. To develop a dashboard for visualising sentiment analysis results.

2.2 Project scope

This project covers the complete development of a sentiment analysis system, from data collection and preprocessing to real-time sentiment classification and storage. The dataset consists of 3,271 English-language Google Maps reviews collected from 13 popular tourist attractions across Malaysia, including Batu Caves, Petronas Twin Towers, Pulau Sipadan, Mossy Forest, Niah National Park, Genting SkyWorld, Taman Nehara and Kek Lok Si Temple.

The sentiment analysis task uses three categories: Positive, Neutral and Negative. To support real-time processing, the system is implemented using a containerised architecture with Docker. Apache Kafka is used for message streaming, while a Convolutional Neural Network (CNN) serves as the primary sentiment classification model. The classified results are stored in Elasticsearch and visualised through Kibana dashboards for monitoring and analysis.

3.0 Data Acquisition and Preprocessing

3.1 Data Sources and Collection

The dataset used in this project was collected from Google Maps reviews of 13 popular tourist attractions in Malaysia. Since Google Maps does not provide a public API for large-scale review extraction, a custom web scraping solution was developed using Selenium with a headless Chrome browser. The scraper automatically accessed the review pages of each tourist attraction, performed dynamic scrolling to load additional reviews and extracted the review text together with the corresponding location URL.

To improve the reliability of the data collection process, several fault-tolerance mechanisms were implemented. Exponential backoff retry logic was used to handle temporary network interruptions. In addition, a progress tracking mechanism was incorporated through a progress.json file, allowing the scraper to resume from the last successfully processed review if the scraping process was interrupted. This approach prevented data loss and reduced the need to restart the collection process.

A total of 3,271 reviews were successfully collected and stored in a structured database containing two attributes: 'place_url' and 'review_text'. After data collection, sentiment labels were generated using the Hugging Face 'cardiffnlp/twitter-roberta-base-sentiment-latest' model. This pre-trained RoBERTa-based transformer model was used to automatically classify each review into one of three sentiment categories: Negative (-1), Neutral (0) and Positive (1). Positive reviews represented the majority class with 2,533 reviews (78%), followed by neutral reviews with 448 instances (14%), while negative reviews had 261 reviews (8%).

3.2 Data Preprocessing

The collected reviews were then processed using several steps to enhance the quality and consistency of the data before being used for model training and deployment. The preprocessing process was built in Python and used in a sequence for each review text.

The first was the text encoding problem, which was fixed with the help of 'ftfy' library to fix malformed Unicode characters and encoding artefacts that are typically added during web scraping. The text was then normalized to lower case for consistency in extracting features and matching tokens.

Then emojis, URLs, special characters and numerical digits were stripped. Since in general, they don't contain many expressive bits of information when it comes to sentiment classification. This cleaned text was then tokenised using the 'word_tokenize()' function from the NLTK library which split each review into a list of word tokens.

After tokenization, English stopwords from the NLTK stopword corpus were filtered out to reduce the number of words that are used in the language but do not carry much semantic value. Then, the rest of the tokens were lemmatized using a 'WordNetLemmatizer' to convert words into their root forms without losing their meaning.

The processed data was stored in a file named 'cleaned_data.csv', which contains two columns: 'clean_review' and 'label'. In the preprocessing phase, 29 reviews were rendered empty after the cleaning and were discarded from the dataset. Thus, a final dataset of 3242 reviews was generated and used to train, test and make a real-time sentiment analysis.

4.0 Sentiment Model Development

This section records the model choice, training process and evaluation for the developed sentiment model, BERT and CNN.

4.1 BERT Model

4.1.1 Model Choice

BERT (Bidirectional Encoder Representations from Transformers) was selected as one of the sentimental analysis models. BERT processes text with a bidirectional attention mechanism, which considers content from both left and right of each token simultaneously, which allows it to capture linguistic meaning of the review text. For this project, bert-base-uncased variant is used, which has a smaller architectural size that consists of 12 layers and 110 million parameters, and a preprocessing step that converts all input text to lowercase and removes accent marks. This variant is suitable for informal text processing in google map review, has better generalisation and reduced memory usage.

4.1.2 Training Process

First, bert-base-uncased was loaded from Hugging Face model hub into google colab. The dataset is divided into training and validation sets with 80:20 ratio with stratified sampling to reduce the possibility of biased evaluation by ensuring the proportions of positive, neutral and negative reviews remain similar in both datasets.

The review texts were then tokenised using BertTokenizer, with a maximum sequence length of 128 tokens. All review texts were padded to 128 tokens and truncated if the length exceeded. The

tokenizer outputs three tensors for each review, `input_ids`, `attention_mask` and `token_type_ids`. The data was then processed in batches of 16 samples.

The model was trained using the AdamW optimiser with a learning rate of 2×10^{-5} . A weight decay value of 0.01 was applied to reduce overfitting. Then, a linear learning rate scheduler with warm-up strategies was used. During the first 10% of the training steps, the learning rate gradually increased from zero to its maximum value and then decreased linearly for the remaining steps. This helps stabilise the fine-tuning process and prevents sudden changes to the pre-trained weights. The model was trained for 4 epochs while implementing a forward pass, loss computation, backpropagation and gradient clipping. Forward pass was performed to generate sentiment predictions from the input reviews, loss computation to measure the difference between the predicted sentiments and the actual labels, backpropagation to calculate the gradients and update the model's parameters so that the prediction errors could be reduced and gradient clipping with a maximum norm of 1.0 was applied to prevent excessively large gradient values, which helps maintain stable training and avoids exploding gradients. After each epoch, the model was evaluated using the test set. If the validation loss was lower than the previous best result, the model was saved as a checkpoint. This approach ensures that the final model selected is the one with the best performance on unseen data rather than simply using the last training epoch.

Finally, the training loss, validation loss and validation accuracy for each epoch were plotted and saved as shown in Figure 4.1 for performance analysis and diagnostic purposes.

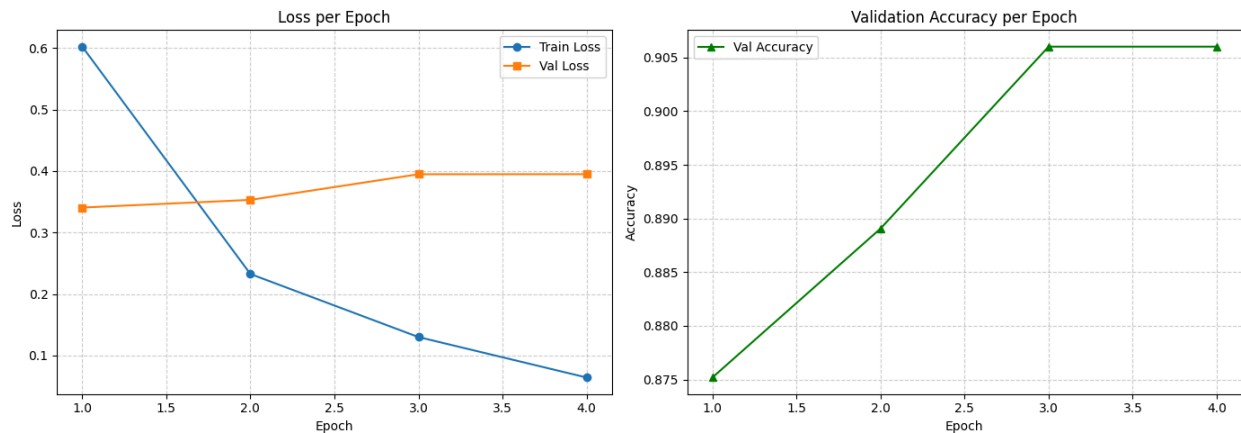


Figure 4.1 Loss and Validation Accuracy per Epoch of BERT sentiment model

4.1.3 Evaluation

The best performing model checkpoint was reloaded using its saved state dictionary and evaluated using a test set containing 649 reviews. The model achieved an overall accuracy of 87.52% as shown in Figure 4.2 below. The report also shows a macro precision of 0.7286, indicating that approximately 73% of the predicted labels for each class were correct on average.

The macro recall was 0.7532, showing that the model correctly identified about 75% of the actual samples in each class. The macro F1-score was 0.7374, suggesting that the model achieved a reasonably balanced performance overall.

The Positive class achieved the best results, with a precision of 0.95, recall of 0.94, and F1-score of 0.95. Out of 507 positive reviews, 478 were correctly classified, while only a small number were misclassified as negative or neutral.

The Negative class showed moderate performance, with a precision of 0.60, recall of 0.75, and F1-score of 0.67. Out of 52 negative reviews, 39 were correctly identified. The relatively high recall indicates that the model was able to detect most negative reviews, although some positive and neutral reviews were incorrectly predicted as negative, resulting in a lower precision.

The Neutral class achieved a precision of 0.64, recall of 0.57, and F1-score of 0.60, which was the lowest among all classes. Out of 90 neutral reviews, only 51 were classified correctly, while the remaining reviews were almost equally misclassified as positive or negative. This indicates that the model had difficulty distinguishing neutral opinions from clearly positive or negative sentiments.

TEST SET METRICS (macro avg)				
Accuracy	: 0.8752			
Precision	: 0.7286			
Recall	: 0.7532			
F1-Score	: 0.7374			
Detailed Classification Report:				
	precision	recall	f1-score	support
Negative	0.60	0.75	0.67	52
Neutral	0.64	0.57	0.60	90
Positive	0.95	0.94	0.95	507
accuracy			0.88	649
macro avg	0.73	0.75	0.74	649
weighted avg	0.88	0.88	0.88	649

Figure 4.2 Classification Report of BERT sentiment model

The confusion matrix presented in Figure 4.3 further highlights the effect of class imbalance in the dataset. Since positive reviews dominate the dataset, the model learned the characteristics of this class particularly well and achieved the highest recall of 0.94. The most common errors occurred when neutral reviews were predicted as either negative or positive, with misclassification rates of approximately 21% and 22%, respectively. This suggests that the model

does not favour one sentiment direction over another but instead struggles to recognise reviews that express neither clearly positive nor clearly negative opinions.

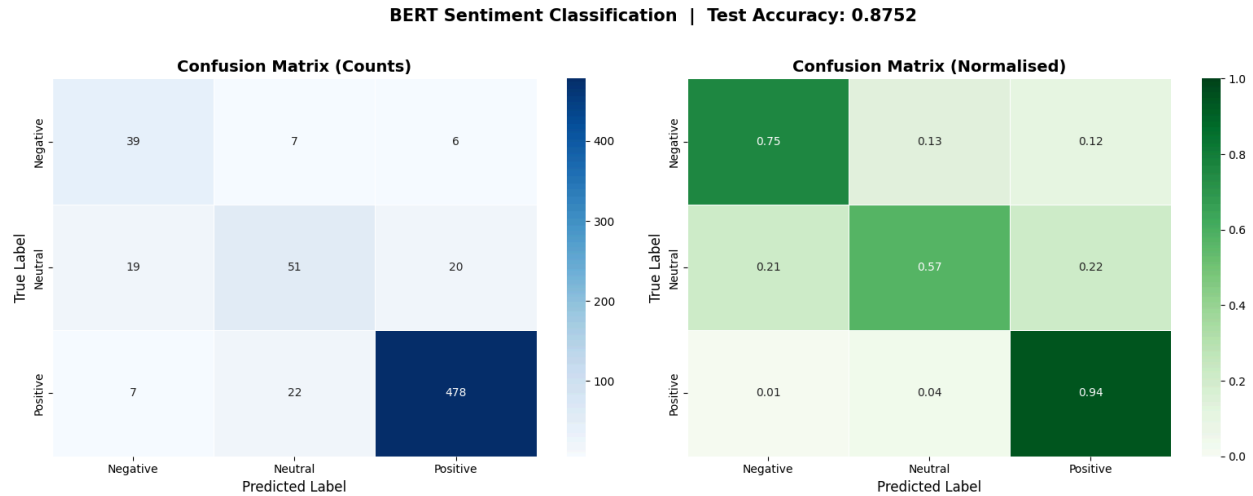


Figure 4.3 Confusion matrix of BERT sentiment model

Overall, the BERT model performed very well on positive reviews and reasonably well on negative reviews. However, the neutral class remains the main limitation of the model. Future improvements could include collecting more neutral reviews or applying techniques such as oversampling and class weighting to reduce the effects of class imbalance and improve the overall macro F1-score.

4.2 CNN Model

4.2.1 Model Choice

As the second sentiment classification model, the CNN was chosen and as the primary model, it was deployed in the real-time pipeline. CNNs are also effective for text classification tasks as the convolutional filters are able to capture local n-gram features from the sequences of token embeddings, which helps the model to capture the sentiment bearing phrases irrespective of their position in the text. A CNN architecture has much fewer parameters than transformer models like BERT, and can use significantly less memory and computation than transformer models, so it is practical for deployment in a streaming pipeline, where the inference process should be performed continuously with low latency.

The architecture used is a single Conv1D layer with 128 filters, kernel size 5, followed by a GlobalMaxPooling1D layer to obtain the most salient feature of the sequence, a Dense layer with 64 units and ReLU activation, a Dropout layer to prevent overfitting with a rate of 0.5, and an output softmax layer with 3 units representing the Negative, Neutral, and Positive categories.

The embedding layer was trained from scratch with vocabulary size up to 10,000 and embedding dimension of 128. The model was lightweight and only has 1.028,483 trainable parameters, which allows it to operate efficiently over CPU for real-time inference.

4.2.2 Training Process

This cleaned dataset of 3,242 reviews was then divided into a training set and a test set in an 80:20 ratio with 2,593 and 649 reviews, respectively. The sentiment labels (-1, 0, 1) were remapped to class indices (0, 1, 2) for Negative, neutral, and Positive, respectively, to meet the requirements of sparse categorical crossentropy of Keras.

Only the training texts were modified with a Keras Tokenizer and used to get an actual vocabulary size of 7,328 unique tokens. All review texts were converted, and then padded and truncated to 128 tokens, based on post-padding/post-truncation.

The model was constructed with the Adam optimiser, learning rate 1×10^{-3} and trained up to 10 epochs using a batch size of 16. During training two callbacks of Keras were employed: ModelCheckpoint only saved model weights when validation loss decreased, and EarlyStopping stopped training when validation loss had not decreased for three consecutive epochs to prevent overfitting. Training was stopped at epoch 5, and the best checkpoint model was saved at epoch 2 with the loss of 0.4435. Below, Figure 4.4 provides a summary of the training by epoch;

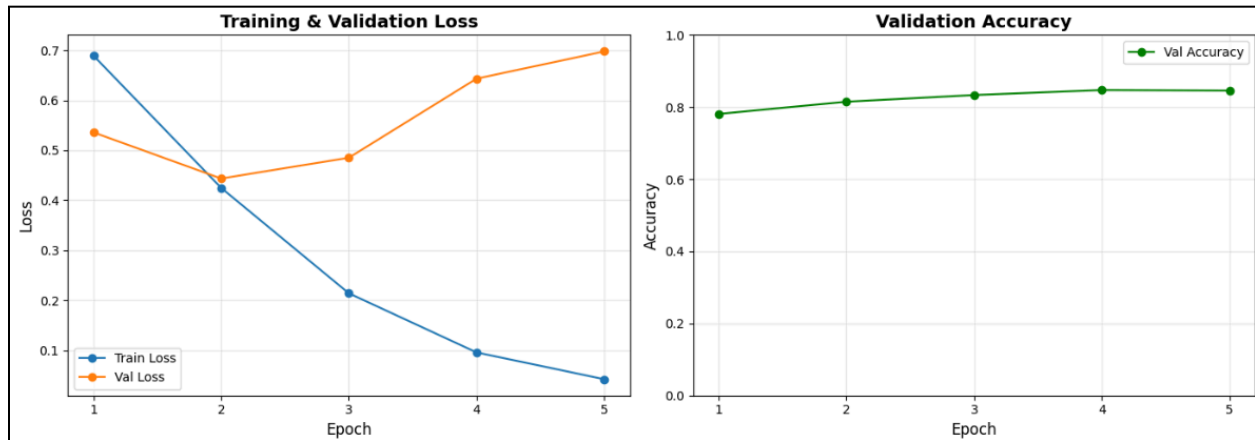


Figure 4.4 Loss and Validation Accuracy per Epoch of CNN sentiment model

As shown in the figure above, the training loss was consistently reduced for each epoch. The validation loss also started to rise from Epoch 3, though the training loss kept decreasing, suggesting that the model could start to overfit the training data from Epoch 3. Early stopping was applied and training was stopped at Epoch 5, as it was supposed to be and the best weights obtained at Epoch 2 were restored. All pipeline deployment was done using the saved checkpoint file `best_cnn_sentiment.keras`.

4.2.3 Evaluation

The model checkpoint was reloaded and tested with the 649 sample test set. The classification report output from Google Colab is presented below in Figure 4.5, and the corresponding confusion matrices are shown in Figure 4.6;

```
Loaded best model from 'best_cnn_sentiment.keras'
```

TEST SET METRICS (macro avg)				
Accuracy	:	0.8151		
Precision	:	0.6566		
Recall	:	0.5289		
F1-Score	:	0.5261		

Detailed classification report:				
	precision	recall	f1-score	support
Negative	0.62	0.10	0.17	52
Neutral	0.45	0.56	0.50	90
Positive	0.89	0.93	0.91	507
accuracy			0.82	649
macro avg	0.66	0.53	0.53	649
weighted avg	0.81	0.82	0.80	649

Figure 4.5 Classification Report of CNN sentiment model

The overall test accuracy of the CNN is 81.51%, while the accuracy of the BERT model is 87.51%. The higher the F1 score, the better. However, due to the large class imbalance in this dataset, the macro-averaged F1 score is 0.5261, indicating that it is hard to classify the minority classes.

The Figure 4.6 below provides details of the per-class performance;

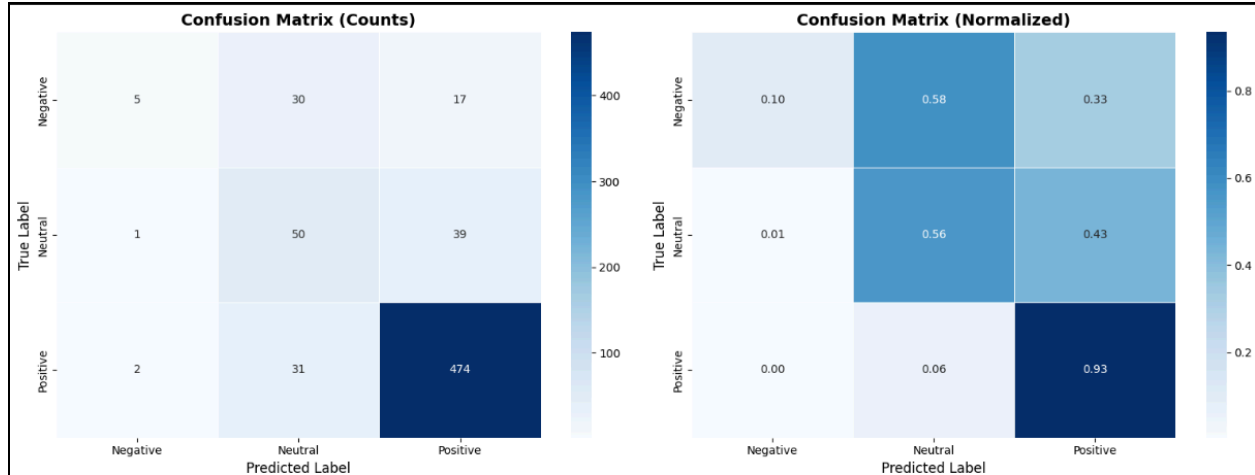


Figure 4.6 Confusion matrix of CNN sentiment model

The Positive class demonstrated the best performance by obtaining a high level of precision of 0.89, recall of 0.93, and the F1-score of 0.91 –reflecting the fact of this class predominance in the training dataset (78%).

The Neutral class demonstrated moderate F1-score of 0.50. Recall was relatively good with 0.56 while precision was low and equal to 0.45 indicating that the model made a lot of mistakes classifying reviews as either positive or negative but neutral.

The Negative class was the worst performing one having the lowest F1-score of 0.17. Even though the precision was relatively high (0.62), the recall was very poor (0.10), thus indicating that the model did not recognize 90% of true negative reviews as negative and predicted them as positive or neutral. It should be noted that such behaviour is explained by the severe class imbalance as the negative class makes only 8% of the whole dataset (261 out of 3,242).

Nevertheless, despite these class-level limitations, the CNN network was chosen for the implementation into the stream processing pipeline due to its small size and fast computation, which are critical requirements for real-time processing. The model performs well on the dominant Positive class (91% F1), which shows that the pipeline can generate relevant outputs for most of the incoming reviews.

4.3 Model Comparison

Table 4.6 summarises the performance of both models across all evaluation metrics, alongside their architectural characteristics and deployment suitability.

Table 4.1 BERT vs CNN Model Comparison

Criteria	BERT	CNN
Accuracy	87.52%	81.51%

Macro Precision	0.7286	0.6566
Macro Recall	0.7532	0.5289
Macro F1-Score	0.7374	0.5261
Parameters	~110 million	1,028,483
Inference Speed	Slow (transformer attention)	Fast (single Conv1D pass)
Memory Usage	High	Low
Deployed in Pipeline	No	Yes

The BERT's bidirectional attention mechanism and pre-trained language representative allow BERT to exceed CNN's performance on all evaluation metrics, especially for the minority Negative and Neutral classes. But BERT's high memory usage and its 110 million parameters made it hard to deploy in real-time use in the Kafka streaming pipeline. The CNN model was thus chosen due to its tradeoff between classification, accuracy, inference speed, and low resource consumption.

5.0 Apache System Architecture

The system has a dual architecture for real-time streaming analytics and batch processing. Cleaned review data is generated from data source and sent to Apache Kafka, where it is distributed to separate processing pipelines. Both pipelines perform sentiment analysis with Convolutional Neural Network (CNN) model, then pass the sentiment scores to Elasticsearch which can be visualised in Kibana. All services are packaged and run in Docker Compose containers. Figure 5.1 shows the system architecture.

Producer Layer

Data ingestion starts with a producer 'producer.py' which uses Kafka. This component loads the review data sequentially from a file 'cleaned_data.csv' and publishes it to a Kafka topic in JSON format. Each message includes 'review ID', 'review text', 'sentiment' and 'time'. A delay between messages of 0.3 seconds is added to simulate the real-world streaming behaviour. The producer also provides fault tolerance features like auto-reconnect after a failure up to 10 times with a delay of 5 seconds re-connecting.

Broker Layer

Apache Kafka is used as a system messaging backbone and Zookeeper coordinates brokers and metadata. The creation of the topics required by Kafka is done automatically during system startup by 'kafka-init', which makes deployment easier and removes the need for manual Kafka topic configuration.

Inference Layer

Sentiment analysis is performed through two separate consumers that implemented the same pre-trained TensorFlow/Keras CNN model 'best_cnn_sentiment.keras'. The streaming consumer 'cnn_stream_consumer.py' will subscribe to the Kafka topic and continuously process the review messages that will be streamed. The reviews are batched every 10 and then tokenised and realigned to a fixed sequence length of 128 tokens. The processed data is then passed into the CNN model to predict sentiment.

The batch consumer 'cnn_batch_consumer.py' does not rely on Kafka and directly reads the entire dataset from 'cleaned_data.csv'. It does not process the records incrementally, but it does inference on the entire dataset on one execution. In this mode, text sequences are standardized to 100 tokens, then predicted. Besides sentiment classification, the batch process captures the execution time, throughput (rows per second), classification accuracy, CPU utilisation and memory usage metrics with the help of the library 'psutil'.

The processing process is the same for both, to ensure consistency in sentiment mapping strategy. The CNN model generates class labels 0,1 and 2 that are mapped to the sentiment -1(negative), 0(neutral) and +1(positive) from the dataset before being stored. The predictions from streaming will be saved in the 'sentiment_results' index in Elasticsearch. While the predictions from batch processing will be saved in two separate indices, 'sentiment_batch' and 'performance_metrics'. These batch indices are built each time they are run to prevent duplicate records from building up across runs.

Storage and Visualisation Layer

All sentiment analysis results are stored in the central storage called Elasticsearch. The data stored in documents are predicted sentiment, classification correctness, throughput and processing timestamps. Kibana integrates with Elasticsearch and provides dashboards for real-time monitoring of system performance and sentiment trends. As reviews are analysed, users can view prediction accuracy, processing throughput and sentiment distribution in real time.

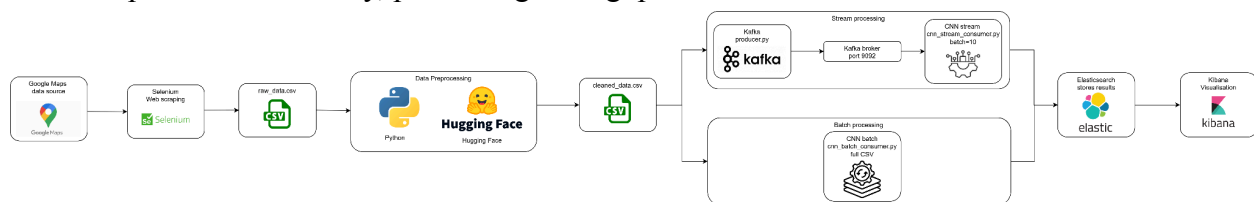


Figure 5.1 System Architecture

6.0 Analysis and Results

6.1 Key Findings

The real-time sentiment analysis pipeline has been successfully working in batch mode and streaming mode. The results have been indexed in the Elasticsearch and visualized using Kibana dashboards. The outputs of the pipeline showed the following key findings;

Sentiment Diffusion: The overall distribution of sentiment across all the records processed by the pipeline was Negative at 47.54%, Positive at 30.53%, and Neutral at 22.11%. This distribution is not representative of the true composition of the original dataset where Positive reviews constitute 78% of all 3,242 samples. The discrepancy comes from two things; first is the stream pipeline was run multiple times during development and testing, resulting in duplicate records in the sentiment_results elasticsearch index, since the streaming consumer does not do any cleanup between runs. Second, the CNN model's recall for the Negative class is disturbingly low at 0.10, leading to a significant number of false negatives and biasing the pipeline-level sentiment counts.

Model Performance: The accuracy average of both pipeline modes, as shown in the Kibana dashboard, is 0.886, higher than the standalone test set accuracy of the CNN (0.8151). This difference is due to the nature of the records in Elasticsearch where the batch pipeline processed all 3,242 records simultaneously, and the streaming pipeline aggregated records from several runs, a weighted combination of both modes.

Batch vs Streaming Throughput: The batch pipeline achieved a peak throughput of about 16 records per second and the streaming pipeline achieved approximately 115 records per second. Batch processing is faster since it loads the entire dataset into memory in one run and does vectorised prediction on all samples at once. The streaming pipeline added latency to each micro-batch because Kafka messages had to be deserialized, JSON parsed and CNN inference had to be performed on groups of 10 messages.

Resource Usage: The batch pipeline run had a CPU utilization of around 95%, which was indicative of the computationally intensive nature of CNN inference on 3,242 samples in parallel. The streaming pipeline has lower and more consistent CPU usage because it processes data incrementally in micro-batches, which distributes the computational load over time.

Most Frequently Discussed Topic: The word cloud visualization showed that the most frequently mentioned words across all the reviews were “beautiful place”, “amazing place”, “good place”, “beautiful view”, “beautiful island”, and “diving paradise”. These terms are consistent with the nature of the data source –Google Maps reviews of Malaysian tourist attractions and the prevalence of positive sentiment in the original labelled dataset.

6.2 Visualization

The Kibana dashboard created for this project consists of 8 panels, each focusing on a different feature of the pipeline output. The dashboard was set up with an absolute time range of June 25, 2026 02:45:00 to June 25, 2026 03:40:00 (MYT) so that all the data collected from two runs of the pipeline would be included in one consistent visualization. The following is a presentation and description of each of the panels.

Figure 6.1 below shows the percentage breakdown of sentiment labels for all records in the `sentiment_results` index in the form of a donut chart. There are 47.64% Negative, 30.35% Positive and 22.11% neutral records. As it was pointed out in section 6.1 above, this breakdown is distorted because of the duplications of the stream records due to several pipeline runs and does not correspond to the actual sentiment breakdown of the original data, where 8% of reviews are Positive.

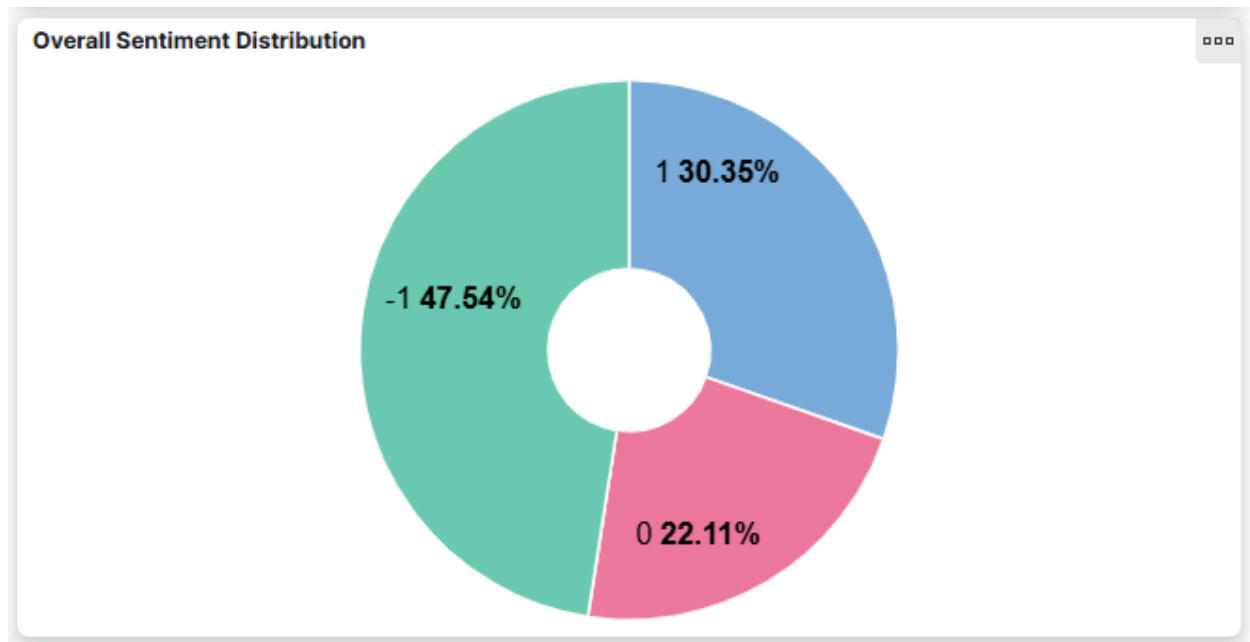


Figure 6.1 Overall Sentiment Distribution

Figure 6.22 below is a panel showing a single-value metric, namely the average accuracy field from the `performance_metrics` index. The index contains average accuracy calculated in real-time by each of the consumers during each run. The average accuracy of 0.886 is obtained after summing up the batch accuracy calculated by `cnn_batch_consumer.py` and streaming accuracy calculated using micro-batches in the streaming consumer. This figure is higher than the standalone test set accuracy of the CNN, which is 0.8151. This is because of the nature of the records created in Elasticsearch after several pipeline runs.

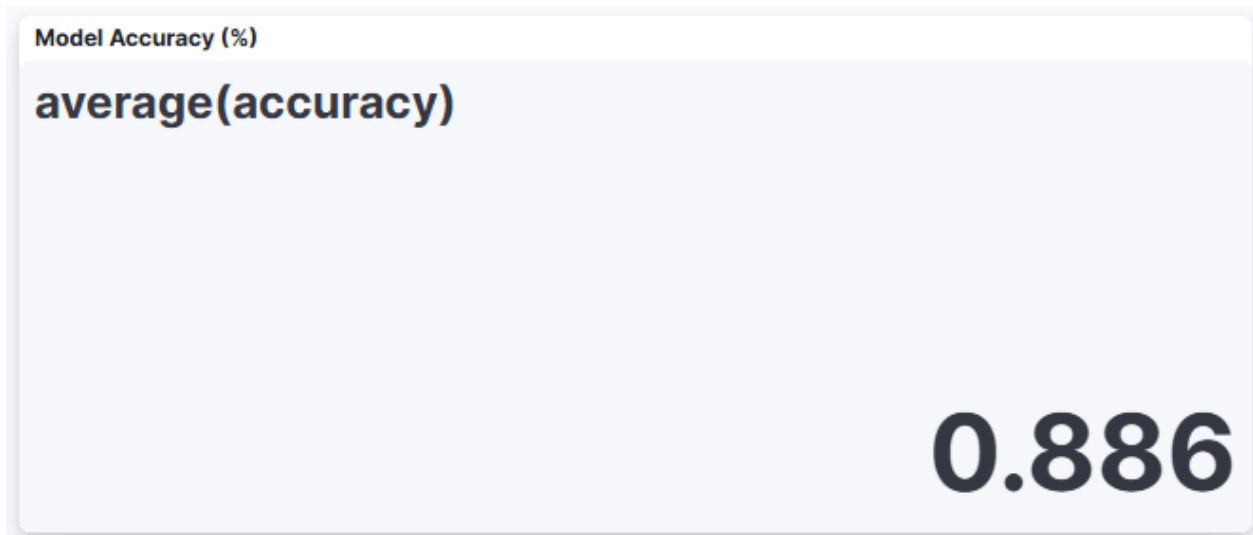


Figure 6.2 Model Accuracy (%)

Then Figure 6.3 below shows a visual representation of the top 10 most common terms in the review_text field. These terms help understand the topics that most interest the tourists visiting Malaysian tourist destinations. Some of the terms include “beautiful place”, “amazing place”, “good place”, “beautiful view”, “beautiful island”, and “diving paradise”. It should be noted that the predominance of the “Other” label is an indication that the majority of values in the review_text field are unique, and no phrases appear more than once in the reviews. This can be explained by the fact that these are natural language reviews.



Figure 6.3 Most Talked About (Tag Cloud)

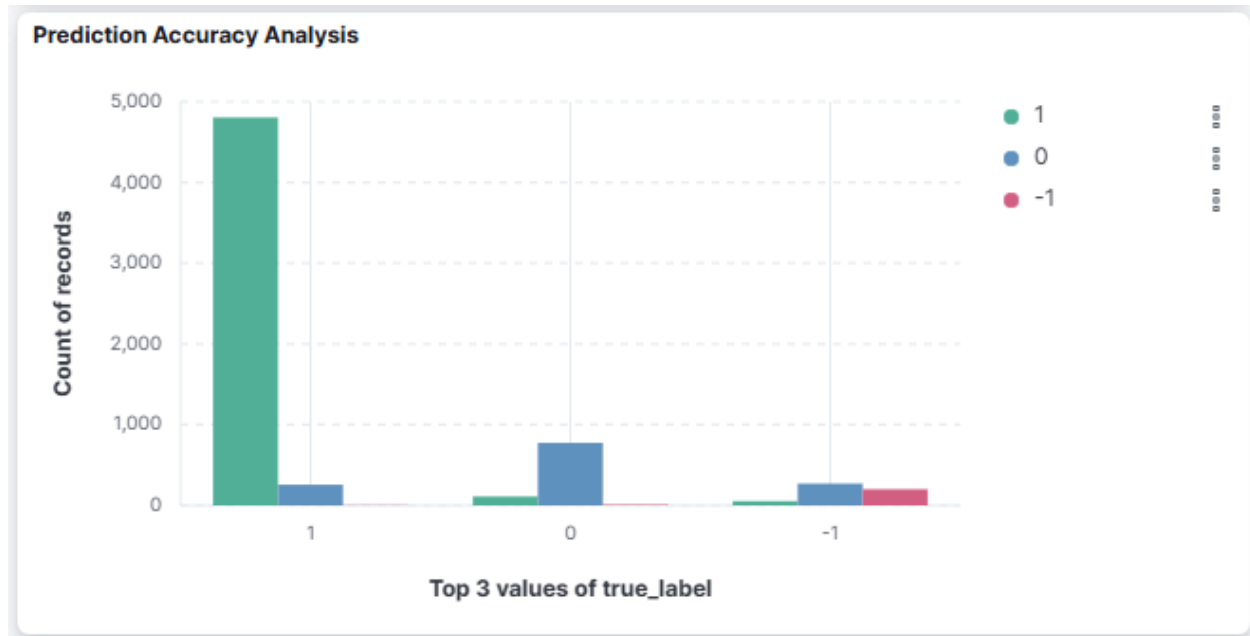


Figure 6.4 Prediction Accuracy Analysis

As shown above, Figure 6.4 is a grouped bar chart depicting the mapping of predicted sentiment versus the true label in each of these three classes, and thus an approximate representation of a confusion matrix of the pipeline. In the case of a true label value of 1 (Positive), the most common predicted sentiment was also 1 (green bar with a total of roughly 4,800 samples), thereby demonstrating the model's excellent recall performance in the positive class. In the case of a true label value of 0 (Neutral), the most common prediction was 0 (blue bar with roughly 750 samples); however, there were other instances that were classified into class 1. In the case of a true label, value of -1 (Negative), predictions were made in all three classes without a definite dominance; this is consistent with the nearly zero negative recall value of the CNN described in Section 4.2.3 before.

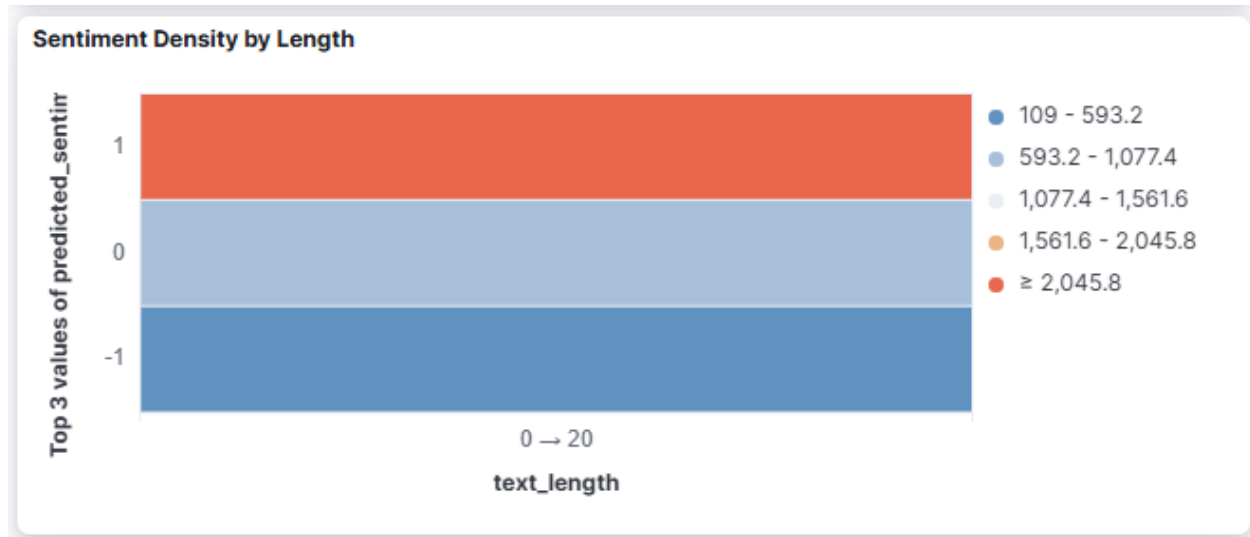


Figure 6.5 Sentiment Density by Length

In Figure 6.5 above, the analysis of the relation between text length and predicted sentiment has been done by using a stacked bar chart. The x-axis has only one bin value between 0 to 20, while all the three sentiments fall under that the `text_length` value in the pipeline was calculated from the `review_text` variable after processing. Due to this, many texts have become very short text values due to stopwords removal, lower casing, and special characters removal.

As for the Figure 6.6 illustrated below, it is shown the line graph for the record count per predicted sentiment class per minute for the absolute time period. It is evident from the line graph that both pipelines exhibited distinct temporal behaviour. There was only a single peak for the batch pipeline, i.e., all 3,242 records had the same timestamp because of the fact that they were recorded at the same time due to a single execution. On the other hand, the streaming pipeline showed a gradual increase in records that lasted for around 16 minutes as the producer was generating messages at an interval of 0.3 seconds. This can be seen in the form of a gradual decline in the line graph on the right side. Label 1 (Positive) was always dominant in both pipelines.



Figure 6.6 Sentiment Trend Over Time

Down here, a bar graph depicting the maximum throughput for both pipeline modes is shown in figure 6.7. The batch pipeline was able to achieve a maximum throughput of about 126 records/second, whereas the streaming pipeline could achieve a throughput of about 115 records/second. Batch execution was able to exhibit a higher throughput because it worked on the full data set in memory in one run of the vectorized CNN inference operation on all 3,242 records in one go. Streaming pipeline, however, had a slightly lower throughput because of the cost associated with consuming Kafka messages, deserializing JSONs, and inference for micro-batches of 10 messages.

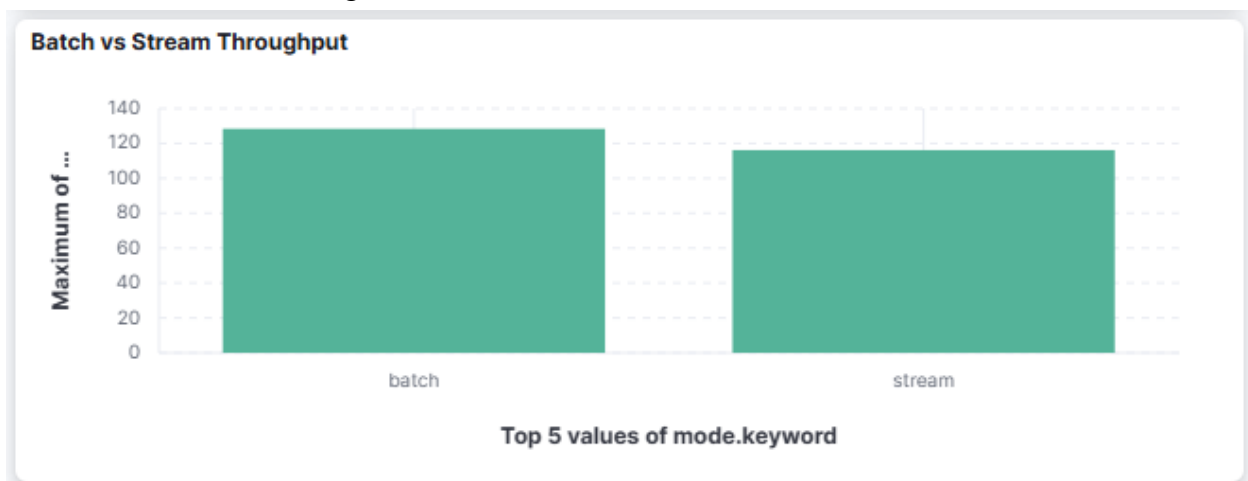


Figure 6.7 Batch vs Stream Throughput

The last chart below here shows Figure 6.8, illustrating the change in CPU usage over time based on the pipeline mode. In the case of batch mode, there was only one point of high CPU usage

close to 95%, indicating the intensity of computations due to the execution of CNN inference for the whole dataset at once. In the streaming mode, the CPU usage was less compared to the previous mode, showing how the micro-batching process spreads out computations over time. It is clear that the operation mode affects the efficiency of the processes. Batch mode provides better throughput, but higher load on system resources.



Figure 6.8 System Resource Usage

6.3 Insights

A few key findings can be gained from the results of the analysis performed on the pipeline outputs;

Tourist sentiment is predominantly positive. The initial dataset, which has labeled the Google Maps reviews, identified 78% out of 3,242 reviews as being positive, implying that the overall sentiment of tourists towards Malaysian tourist spots is quite satisfactory. The result is backed up by the tag cloud's prevalence of terms such as “beautiful place”, “amazing experience”, and “diving paradise”.

Class imbalance is the primary limitation of the CNN model. The macro F1 score of 0.5261 for the CNN model is much less than its accuracy of 0.8151, which indicates that accuracy alone can be an inadequate measure of evaluation for this kind of imbalance dataset. The very low recall for the Negative class (0.10) implies that the system is incapable of detecting negative reviews in the real world. This problem needs to be addressed in the next iteration using any of the three methods; oversampling of the minority classes, class-weight adjustment during training, or replacement with a more capable model such as DistilBERT.

Batch processing suits fixed-volume retrospective analysis. The batch process completed the analysis of the full 3,242 records dataset in a single pass at a throughput rate of about 126 records/second. However, batch processing does not allow real-time monitoring as all records have the same timestamp, hence no meaningful time-trend analysis can be done.

Streaming processing enables real-time monitoring with manageable overhead. The streaming pipeline generated a slightly reduced throughput of about 115 messages per second because of message deserialization and micro-batch overhead. Nevertheless, the pipeline was able to generate time-distributed records that allowed for generation of visualization trends in sentiments on Kibana, proving the benefit of streaming for live monitoring use cases where real-time insights are important, but high throughput is not.

Duplicate records limit the reliability of Kibana aggregations. Since the stream consumer does not delete the previous records before running the pipeline again, each run the pipeline appends all records into Elasticsearch. Consequently, the sentiment distribution and number of records on the Kibana dashboard were based on several runs of the pipeline instead of just one. In practice, the implementation of de-duplication or index rotation should be considered to maintain the dashboard results.

7.0 Optimization and Comparison

This section proposes several optimizations to improve the sentiment classification model and the overall system architecture.

The first strategy that can be applied is **knowledge distribution and model quantisation** by training a smaller transformer model with BERT which is DistilBERT using the fine tuned transformer model. This optimisation aims to address the high computational and memory requirements of BERT, which prevented it from being deployed in this real-time sentiment analysis pipeline despite achieving the best classification performance. By reducing the model size and computational complexity, the system can achieve faster inference speeds and lower resource consumption while maintaining most of the original model's classification capability.

Kafka Partitioning and Consumer Group Scaling can be implemented to increase the scalability and reliability of the streaming pipeline, by dividing Kafka topic into multiple partitions and processing it using several consumer instances within the same consumer group. The current implementation relies on a single consumer instance, which limits system throughput and creates a single point of failure. By processing messages in parallel and automatically redistributing workloads when a consumer fails, the system can support larger streaming workloads and provide higher availability.

Lastly, **Elasticsearch Bulk Indexing and Asynchronous Processing** can be applied to improve data ingestion performance. Instead of sending each prediction result individually to Elasticsearch through synchronous requests, the streaming consumer can temporarily accumulate multiple predictions and submit them as a single bulk request. Current arrangement of individual indexing requests generate significant network overhead and increase indexing latency when

large volumes of data are processed. By using the Bulk API, communication overhead is reduced and enables Elasticsearch to ingest sentiment data more efficiently, resulting in faster dashboard updates and improved real-time analytics.

Table 7.1 Comparison Between Baseline and Optimised Architecture

Technical Aspect	Baseline Architecture	Optimised Architecture	Expected Result
Sentiment Model	FP32 TensorFlow/Keras CNN model deployed for both streaming and batch inference.	DistilBERT model deployed using ONNX Runtime.	Lower memory consumption, faster inference, better classification performance.
Kafka Configuration	Single topic partition with one streaming consumer instance.	Multiple Kafka partitions processed by a consumer group.	Higher throughput through parallel processing and improved scalability.
Elasticsearch Indexing	Synchronous indexing requests from the streaming consumer.	Asynchronous micro-batching using the Elasticsearch Bulk API.	Reduced network overhead and improved indexing throughput.

8.0 Conclusion and Future Work

The project successfully created a real-time pipeline for the sentiment analysis of Google Maps reviews of Malaysian tourist spots, employing Apache Kafka, the CNN-based sentiment classifier, Elasticsearch, and Kibana in a unified pipeline.

In total, 3,242 English reviews were scraped from 13 tourist spots around Malaysia via Selenium-based web scraping and annotated with the help of the pre-trained cardiffnlp/twitter-roberta-base-sentiment-latest RoBERTa model. Two sentiment models were trained and tested, specifically BERT with an accuracy of 87.52% and macro F1-score of 0.7374, and CNN with an accuracy of 81.51% and macro F1-score of 0.5261. The BERT model showed better classification performance, especially regarding the minority classes of Negative and Neutral sentiments, owing to bidirectional contextual representations. Nonetheless, the CNN model was chosen for the pipeline implementation because of its small size of approximately one million parameters, high inference speed, and small memory consumption, which are all necessary properties for real-time processing.

The pipeline using `cnn_stream_consumer.py` and Apache Kafka for streaming achieved a throughput rate of about 115 records per second. The pipeline using `cnn_batch_consumer.py` for batch processing achieved a throughput of about 126 records per second. Both streams have their

outputs saved to Elasticsearch and were also visualized on a Kibana dashboard made up of eight panels that analyze sentiment distribution, accuracy, streaming versus batch throughput, resource consumption, and sentiment trends.

The system showed the possibility of implementing a lightweight deep learning model in a Kafka stream environment and visualization of sentiment insights in an interactive way. The Kibana dashboard revealed that tourist reviews about Malaysia attractions are generally positive reviews that include descriptions such as beautiful scenery, unique natural environment, and memorable experience, which is consistent with the 78% positive proportion found in the labeled dataset.

There were several limitations observed. For instance, the macro recall of the CNN model for the Negative class is low at 0.10, which means that the pipeline is unreliable for identifying negative reviews because of class imbalance. The problem with duplicate records appeared within the Elasticsearch indexing as a result of the accumulation of duplicated data during several iterations of the streaming pipeline. Moreover, there is no possibility to use BERT for real-time streaming analysis because of its excessive resource consumption.

The following suggestions can be offered for future work. Firstly, it is necessary to replace the CNN model with DistilBERT –an efficient distilled transformer that possesses 97% of BERT’s effectiveness but requires 40% less model size and 60% less inference time. Secondly, class imbalance can be resolved using SMOTE oversampling or class-weighted loss functions. Thirdly, it is necessary to optimize Elasticsearch index management by implementing the process of deduplication or index rotation. Fourthly, the scalability of Kafka partitioning and consumer groups can be investigated for increasing streaming performance. Finally, the number of platforms for data collection can be increased to TripAdvisor and [Booking.com](https://www.booking.com) and also data can be collected on Malay language reviews.

Appendix

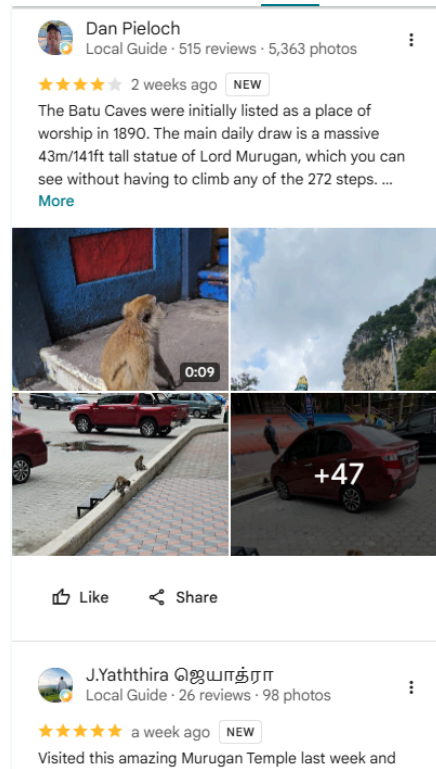


Figure 1 Example reviews of tourist attractions in Malaysia